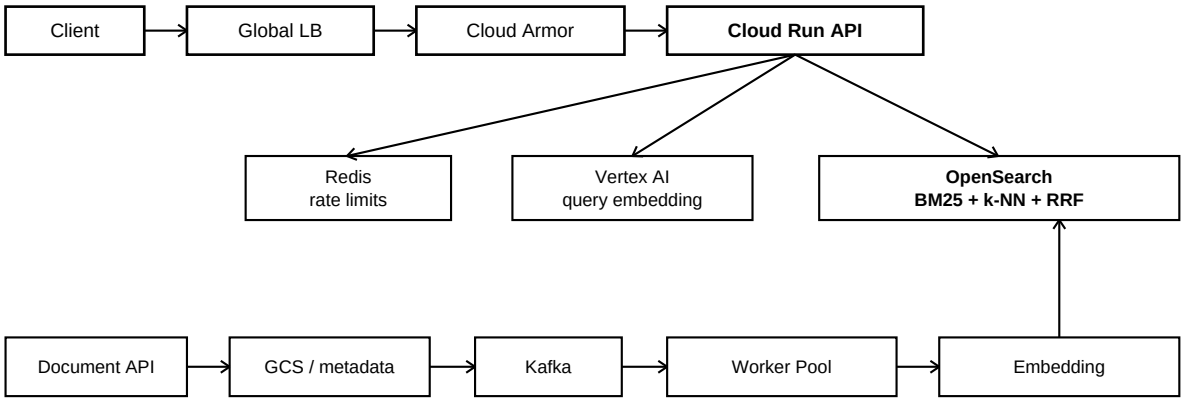
Distributed Document Search Service

Architecture Design - Spring Boot / Cloud Run / OpenSearch / Kafka

1. Requirements

Functional	Non-functional
Ingest, update and delete tenant documents. Hybrid full-text search with filters and ranked top-K. Tenant-isolated reads/writes. Asynchronous indexing through a stable REST API.	10M+ documents; 1,000+ searches/sec; p95 <= 500 ms. Horizontal scale and fault tolerance. OAuth2, OWASP edge protection, API rate limits. Near-real-time search with observable freshness.

2. Architecture



Indexing is asynchronous; versioned Kafka events; retries -> DLT. Search and indexing scale independently.

Edge and security. A Global External Application Load Balancer terminates the public path. Cloud Armor applies preconfigured OWASP protections and coarse edge throttling. Spring Security validates OAuth2/JWT bearer tokens and derives `tenant_id` from a trusted claim. Redis supplies per-tenant application quotas. Cloud Run ingress is restricted to load-balancer/internal traffic.

Multi-tenancy. Small tenants share pooled OpenSearch indexes. Each request carries the tenant as both a mandatory non-scoring filter and the OpenSearch routing key. Routing limits shard fan-out; it is not authorization. Large/hot or regulated tenants can move to dedicated indexes/clusters through a tenant-placement catalog without changing the external API.

3. Hybrid Search: BM25 + Dense Vectors + RRF

Why hybrid. BM25 is strong for exact terms, IDs and rare vocabulary but can miss semantic equivalents such as *automobile* and *car*. Dense embeddings represent conceptual meaning and increase recall when query and document vocabulary differ. The service keeps both signals rather than replacing lexical retrieval.

Index time	title/content -> Lucene BM25 structures	title + content -> RETRIEVAL_DOCUMENT embedding -> knn_vector
Query time	multi_match(title^3, content)	RETRIEVAL_QUERY embedding -> filtered k-NN
Fusion	BM25 rank list	Vector rank list -> OpenSearch RRF search pipeline

Vector index. OpenSearch stores a dense `knn_vector` and uses Faiss/HNSW with cosine similarity. Tenant and metadata filters are placed inside the k-NN clause so semantic candidates obey the same isolation constraints as lexical candidates. Embeddings are generated asynchronously for documents and synchronously for queries.

RRF. Raw BM25 and cosine/vector scores are not directly comparable because they measure different quantities and have different distributions. Instead of adding raw scores, Reciprocal Rank Fusion combines the rank position from each result list:

```
RRF(d) = sum [ 1 / (k + rank_i(d)) ]
```

The prototype uses OpenSearch's `score-ranker-processor` with `rank_constant=60`. A score-normalized weighted blend is another valid approach, but RRF is a clean baseline because it does not require score calibration. Relevance tuning should use a judged query set (for example NDCG@10 / Precision@10) rather than arbitrary weights.

4. Data Flows and Consistency

Indexing. `POST /v1/documents` authenticates the tenant, persists the durable source/metadata in production, publishes a versioned Kafka event and returns 202. Worker-pool consumers generate the document embedding and use the OpenSearch Bulk API. Kafka delivery is at least once; stable document IDs, tenant routing and external versions make retries idempotent. Persistent failures move to a dead-letter topic.

Search. Validate JWT -> enforce tenant quota -> resolve placement -> generate/reuse query embedding -> send one routed OpenSearch hybrid query -> BM25 and filtered ANN retrieve candidates -> RRF fuses -> fetch top-K. Search is near-real-time/eventually consistent because accepted writes need consumer processing, embedding, indexing and OpenSearch refresh before they become visible.

5. Scalability, Reliability and Operations

• **API:** stateless Cloud Run service scales by request concurrency/CPU; minimum warm instances protect p95 tail latency.

• **Indexer:** Cloud Run worker pool scales independently from search. Kafka partitions absorb bursts and allow parallel consumers; consumer lag is the scaling signal.

• **OpenSearch:** primary shards distribute storage and work; replicas provide failover and additional searchable copies. Tenant routing reduces scatter/gather. Initial shard count comes from measured indexed bytes and load tests, not document count alone.

• **Hybrid cost:** query embeddings and ANN add critical-path work. Keep the embedding endpoint and search cluster close to the API, cap vector candidate depth, use bounded timeouts, and cache genuinely hot query embeddings. Validate p95 under simultaneous indexing.

• **Failure handling:** Kafka buffers OpenSearch/embedding outages; retries are bounded; DLT isolates poison records. OpenSearch remains a derived index and can be rebuilt from durable source/event history.

• **Caching:** Redis holds rate limits, placement data and optional hot query-embedding/results. OpenSearch/Lucene caches and OS page cache handle repeated search structures.

6. Observability and Service Objectives

Cloud Logging, Cloud Monitoring, Trace/OpenTelemetry and Actuator/Micrometer track API latency/errors, OAuth failures, tenant throttling, embedding latency/errors, Kafka lag/DLT rate, OpenSearch lexical/vector query latency, indexing latency, cluster health/JVM/disk and Redis health. Alerts should include SLO error-budget burn.

Objective	SLI	Target
Search latency	End-to-end /v1/search duration	p95 <= 500 ms; internal objective <= 450 ms
Availability	Successful eligible requests / total	SLO 99.95%; SLA 99.9% monthly
Index freshness	Accepted document becomes searchable	99% within 10 seconds
Search errors	5xx / eligible search requests	< 0.1% steady state

7. API Surface and Prototype Scope

`POST /v1/documents` -> 202; `DELETE /v1/documents/{id}?version=n` -> 202; `POST /v1/search` -> ranked top-K; `GET /actuator/health` -> readiness/health. Result size is bounded; deep pagination should use PIT + search_after rather than large offsets.

The repository uses one Spring Boot/Gradle image with standard package `com.example.search`. The API runs as a Cloud Run service; the Kafka consumer runs as a Cloud Run worker pool. Docker Compose provides OpenSearch 3.8, Kafka and Redis. Local mode uses deterministic vectors only to exercise the hybrid/RRF plumbing; production mode uses Vertex AI embeddings.